

1) Write a Verilog code for ALU using Function and Task

```
p 10, 2025 NOW Resources Community Help Playgrounds Profile
1 // Code your design here
2 module ALU(input [3:0] a, b, input [2:0] sel, output reg
  [4:0] y);
3
4 // Function for addition
5 function [4:0] add;
6   input [3:0] x, z;
7   begin
8     add = x + z;
9   end
10 endfunction
11
12 // Task for subtraction
13 task sub;
14   input [3:0] x, z;
15   output [4:0] result;
16   begin
17     result = x - z;
18   end
19 endtask
20
21 always @(*) begin
22   case(sel)
23     3'b000: y = add(a,b); // addition
24     3'b001: sub(a,b,y); // subtraction
25     3'b010: y = a & b; // AND
26     3'b011: y = a | b; // OR
27     3'b100: y = a ^ b; // XOR
28     default: y = 5'b00000;
29   endcase
30 end
31 endmodule
32
```

2) Write Verilog Code for demux 1*8 using compiler directives

```

1 // Code your design here
2 `define WIDTH 8
3
4 module demux1x8(input din, input [2:0] sel, output reg
  [WIDTH-1:0] y);
5     always @(*) begin
6         y = 8'b0;
7         y[sel] = din;
8     end
9 endmodule
10
11
12

```

3) Write Verilog Code for full adder the circuit diagram using parameter

```

1 // Code your design here
2 module full_adder #(parameter N=1)
3     (input [N-1:0] a, b, input cin, output [N-1:0] sum,
4      output cout);
5     assign {cout, sum} = a + b + cin;
6 endmodule
7

```

4) Write Verilog code for swapping two variables using blocking and non blocking assignments.

```

1 // Code your design here
2 module swap;
3     reg [7:0] a, b;
4
5     initial begin
6         a = 5; b = 10;
7         $display("Before swap: a=%d b=%d", a, b);
8
9         // Blocking assignment
10        a = a ^ b;
11        b = a ^ b;
12        a = a ^ b;
13        $display("After blocking swap: a=%d b=%d", a, b);
14
15        // Non-blocking assignment
16        a <= b;
17        b <= a;
18        #1 $display("After non-blocking swap: a=%d b=%d", a, b);
19    end
20 endmodule
21

```

5) Write Verilog code for updowncounter using loops.

```

1 // Code your design here
2 module updown;
3     integer i;
4     reg [3:0] cnt;
5
6     initial begin
7         $display("Up Counter:");
8         for(i=0;i<10;i=i+1) begin
9             cnt = i;
10            #1 $display("%d", cnt);
11        end
12
13        $display("Down Counter:");
14        for(i=9;i>=0;i=i-1) begin
15            cnt = i;
16            #1 $display("%d", cnt);
17        end
18    end
19 endmodule
20 |
21

```

6) Write a Verilog code for implementing PIPO(PARALLEL IN PARALLEL OUT).

```

1 // Code your design here
2 module PIPO(input [3:0] din, input clk, output reg [3:0] dout);
3     always @(posedge clk) begin
4         dout <= din;
5     end
6 endmodule
7 |
8
9

```

7) Write a Verilog code for generating 100 random numbers

```

1 // Code your design here
2 module random_gen;
3     integer i;
4     initial begin
5         for(i=0;i<100;i=i+1)
6             $display("Random[%0d] = %0d", i, $random);
7     end
8 endmodule
9 |
10
11
12

```

8) Write a task to add two numbers with a delay

```

1 // Code your design here
2 module task_add;
3     task add_delay;
4         input [3:0] a,b;
5         output [4:0] sum;
6         begin
7             #5 sum = a+b;
8         end
9     endtask
10
11     reg [3:0] x,y;
12     reg [4:0] s;
13
14     initial begin
15         x=5; y=7;
16         add_delay(x,y,s);
17         $display("Sum after delay = %d", s);
18     end
19 endmodule
20 |
21
22
23

```

9) Write a Verilog code for Sequence detector for detecting the sequence 1010.

```

1 // Code your design here
2 module seq1010(input clk, reset, din, output reg detect);
3     reg [3:0] state;
4
5     always @(posedge clk or posedge reset) begin
6         if(reset) begin
7             state <= 0; detect <= 0;
8         end else begin
9             case(state)
10                0: state <= (din)?1:0;
11                1: state <= (din)?1:2;
12                2: state <= (din)?3:0;
13                3: state <= (din)?1:4;
14                4: begin detect <= 1; state <= 0; end
15                default: state <= 0;
16            endcase
17        end
18    end
19 endmodule
20
21
22

```

10) Write a Verilog code that uses a task to generate stimulus for a 4-bit counter testbench. The task should apply reset and toggle the clock.

```

1 // Code your design here
2 module counter4(input clk, reset, output reg [3:0] q);
3     always @(posedge clk or posedge reset) begin
4         if(reset) q <= 0;
5         else q <= q + 1;
6     end
7 endmodule
8 module tb;
9     reg clk, reset;
10    wire [3:0] q;
11    counter4 c1(clk, reset, q);
12    task init;
13        begin
14            reset = 1; clk = 0;
15            #5 reset = 0;
16        end
17    endtask
18    task toggle_clk;
19        integer i;
20        begin
21            for(i=0;i<20;i=i+1) begin
22                #5 clk = ~clk;
23            end
24        end
25    endtask
26    initial begin
27        init;
28        toggle_clk;
29        $finish;
30    end
31 endmodule
32
33

```